

Corrigé examen 2017-12 L3 Miage Bases de données avancées

1 Interrogation

Plus au programme en 2020 ☺

2 Indexation

1. Il s'agit d'un arbre B+. Attention de bien recopier l'élément de débordement à gauche, et de chaîner les feuilles entre elles. Lors des débordements de nœuds internes, le comportement est le même que pour l'arbre B.

Il vaut mieux indiquer les étapes intermédiaires (par exemple chaque débordement) : cela nous permet de vous trouver des points en cas d'erreur sur le résultat final.

Ordre 2: au plus 4 éléments par nœuds. L'ordre ici est l'ordre alphabétique.

Ceci est un arbre difficile :(

ghinp : débordement

i
/ \

ghi -> nop **qr** : débordement

i p
/ | \

deg hi -> nop -> **qr** : débordement

g i p
/ | | \

cdeg -> hi -> Lmnop -> **qr** : débordement

g i n p
/ | | | \

cdefg -> hi -> Lmn -> op -> **qr** : débordement

e g i n p : débordement (comme arbre B)

```

      /   |   |   |   \
cde -> fg -> hi  -> Lmn -> op -> qr

      i
      /       \
    e   g       n   p
  /   |   \   /   |   \
acde -> fg -> hi  -> jkLmn -> op -> qr :débordement

```

```

      i
      /       \
    e   g       L   n   p
  /   |   \   /   |   |   \
abcde -> fg -> hi  -> jkL-> mn -> op -> qr :débordement

```

```

      i
      /       \
    c e   g       L   n   p
  /   |   |   \   /   |   |   \
abc -> de -> fg -> hi  -> jkL-> mn -> op -> qr

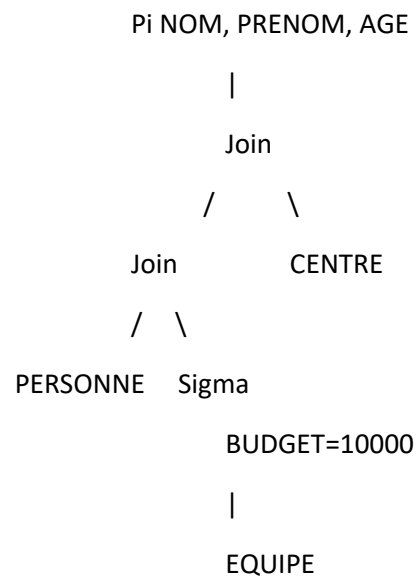
```

Fin.

2. Il faut 11 pages pour stocker cet arbre.
3. La racine, car toute recherche commence par charger cette page. C'est la plus fréquemment utilisée.

3 Optimisation))

1. Algèbre: $\Pi \text{ NOM, PRENOM, AGE}((\text{PERSONNE join } (\sigma \text{ BUDGET}=10\ 000 (\text{EQUIPE}))) \text{ join } \text{CENTRE})$



2. Notation ORACLE EXPLAIN

0 SELECT

1 NESTED LOOP

2 NESTED LOOP

3 TABLE ACCESS FULL PERSONNE

4* TABLE ACCESS FULL EQUIPE

5 TABLE ACCESS FULL CENTRE

4 FILTER BUDGET = 10 000

3. Arbre B+ sur EQUIPE(BUDGET) pour faire rapidement BUDGET=10000
4. Eventuellement B+ sur PERSONNE(ID) (devrait s'appeler PERSONNE.IDEQUIPE) pour trouver rapidement celle correspondant à l'équipe, idem pour CENTRE(IDC)

Notation ORACLE EXPLAIN

0 SELECT

1 NESTED LOOP

2 NESTED LOOP

3 TABLE ACCESS BY ROWID EQUIPE

4 INDEX RANGE SCAN ID_EQUIPE

```

5      TABLE ACCESS BY ROWID PERSONNE
6      INDEX RANGE SCAN IDEQUIPE_PERSONNE
7      TABLE ACCESS BY ROWID CENTRE
8      INDEX UNIQUE SCAN CENTRE
-----

```

3 FILTER BUDGET= 10000

5 PERSONNE.EQUIPEID= EQUIPE.ID

7 CENTRE.ID = EQUIPE.IDC

5. On peut indexer EQUIPE par hachage sur budget, car c'est une recherche par égalité (=10000).

Pour les petites malines et les petits malins: on peut observer que la requête ne se sert pas du tout de CENTRE (le select ne fait rien avec ses attributs). Donc si on suppose que chaque équipe a forcément un centre, on peut dégager cette jointure ! (bonus bonus 😊)

4 Sériabilité et recouvrabilité

1. Au moment de a1, le SGBD annule toutes les opérations de la transactions 1 (qui n'a fait qu'une lecture, sans effet sur la bd). La transaction 2 peut continuer (elle ne lit rien fabriqué par T1).
2. Comme au 1) (y n'est pas fabriqué par T1).
3. L'histoire n'est pas en série car par exemple w1[y] est entre 2 opérations de T2 (r2[x]w1[y]...r2[y]). Les opérations sont donc entremêlées.
4. On énumère les conflits (on peut construire le graphe de sériabilité en même temps):

w1[y]-r2[y], w3[z]-w1[z], w3[z]-w2[z], w1[z]-w2[z]

T3-> T1 -> T2

|-----^

Le graphe de sériabilité est acyclique. Par application du théorème de sériabilité, on peut en déduire que cette histoire est sérialisable (et équivalente à T3;T1;T2)

5. T2 lit y de T1, mais T2 valide avant T1. Donc l'histoire n'est pas recouvrable.

Comme stricte => éac => recouvrable, on peut en déduire qu'elle ne peut pas être éac ni stricte.

6. H^r: r₂[x]w₁[y]w₃[z]w₁[z]r₃[x]C₃r₂[y]r₁[x]w₂[z]C₂C₁

Histoire exécutée: r₂[x]w₁[y]w₃[z]r₃[x]C₃

X R : T2 T3

Y W : T1

Z W : T3

Z W : T1 bloquée

Histoire exécutée: $r_2[x]w_1[y]w_3[z]r_3[x]C_3w_1[z]$

Au moment du commit de T3 C_3 : T1 redémarre

X R : T2

Y W : T1

Z W : T1

Histoire exécutée: $r_2[x]w_1[y]w_3[z]r_3[x]C_3w_1[z] r_1[x] C_1$

Y W : T1

Y R : T2 bloquée

Z W : T1

Z W : T2 en attente

X R : T2, T1

C2 : T2 en attente

Histoire exécutée: $r_2[x]w_1[y]w_3[z]r_3[x]C_3w_1[z] r_1[x] C_1 r_2[y] w_2[z] C_2$

Au moment du commit de T1 C_1 : T2 redémarre (y et w libérés)

7. (exo dur dur !)

Pour être plus clair, le sujet devrait dire "comportant des transactions différentes".

Sinon c'est clairement faux :

H1: $w_1[x]-w_2[x]$ H2: $w_2[x]-w_1[x]$

H1 sérialisable, H2 sérialisable, mais le graphe de H1;H2 a un cycle, donc pas sérialisable.

Ce sont donc des transactions distinctes. Le seul pb c'est les variables communes.

Par l'absurde: supposons que H1;H2 ne soit pas sérialisable. Donc, il existe un cycle dans le graphe de sérialisabilité. Si le cycle ne parcourt pas de nœuds des transactions de H2, alors c'est qu'il est dans H1, ce qui n'est pas possible, car H1 est sérialisable.

Soit donc le nœud de H2 dans ce cycle. Comme H1 est avant H2, tous les conflits possibles entre transactions de H1 et de H2 vont de H1 vers H2. Donc de H2 on ne peut pas rejoindre H1. Donc le cycle est entièrement dans H2. Ce qui n'est pas possible, car H2 est sérialisable. Donc H1;H2 est nécessairement sérialisable.